

SQL Dinámico



Objetivo: Explorar el uso de cursores y transacciones para el control detallado sobre manipulaciones de datos, y entender Grant y Revoke como herramientas para administrar permisos de acceso a datos. Introducir el concepto de triggers, esquemas y sinónimos y su aplicación en BD..



SQL a fondo... Dominando consultas avanzadas y técnicas dinámicas

REPASANDO PROCEDIMIENTOS



```
-- 1) Procedimiento almacenado - Control de pedidos por cliente
-- Objetivo: listar pedidos de un cliente y mostrar un mensaje según la cantidad.
CREATE PROCEDURE sp_AnalizarPedidosCliente
    @ID_Cliente INT
AS
BEGIN
    DECLARE @TotalPedidos INT;
    SELECT @TotalPedidos = COUNT(*)
    FROM C09Pedidos
    WHERE ID_Cliente = @ID_Cliente;
    PRINT 'Cliente: ' + CAST(@ID_Cliente AS VARCHAR) +
        ' - Pedidos encontrados: ' + CAST(ISNULL(@TotalPedidos,0) AS VARCHAR);

    PRINT
        CASE
            WHEN @TotalPedidos = 0 THEN 'El cliente no tiene pedidos registrados.'
            WHEN @TotalPedidos = 1 THEN 'El cliente tiene un único pedido.'
            ELSE 'El cliente tiene múltiples pedidos.'
        END;

    RETURN @TotalPedidos;
END;
GO
```

REPASANDO PROCEDIMIENTOS



```
-- 2 Procedimiento almacenado - Calcular total y validar stock
-- Objetivo: calcular el total de un pedido y mostrar advertencias si el monto supera cierto umbral.
CREATE OR ALTER PROCEDURE sp_TotalPedido
    @ID_Pedido INT
AS
BEGIN
    DECLARE @Total DECIMAL(10,2);
    SELECT @Total = SUM(Cantidad * PrecioUnitario)
    FROM DetalleC09Pedidos
    WHERE ID_Pedido = @ID_Pedido;

    PRINT 'Total del Pedido ' + CAST(@ID_Pedido AS VARCHAR) + ': $' + CAST(@Total AS VARCHAR);
    IF @Total > 1000
        PRINT '⚠ Pedido de alto valor. Verificar autorización.';
    ELSE
        PRINT 'Pedido dentro del rango habitual.';

    RETURN 0;
END;
GO
```

```
EXEC sp_TotalPedido @ID_Pedido = 2
```

REPASANDO FUNCIONES



```
-- 3 Función escalar - Determinar categoría de producto
-- Objetivo: clasificar productos según su precio.
CREATE FUNCTION fn_CategoriaProducto(@Precio DECIMAL(10,2))
RETURNS VARCHAR(20)
AS
BEGIN
    DECLARE @Categoria VARCHAR(20);

    SET @Categoria = CASE
        WHEN @Precio >= 1000 THEN 'Premium'
        WHEN @Precio BETWEEN 500 AND 999 THEN 'Medio'
        ELSE 'Económico'
    END;
    RETURN @Categoria;
END;
```

```
SELECT NombreProducto, PrecioUnitario,
dbo.fn_CategoriaProducto(PrecioUnitario) AS Categoria
FROM C09Productos;
```

REPASANDO FUNCIONES



```
-- 4 Función con valor de tabla en línea (iTVF)
-- Objetivo: obtener los pedidos realizados en un año determinado.
CREATE FUNCTION fn_PedidosPorAnio(@Anio INT)
RETURNS TABLE
AS
RETURN (
    SELECT p.ID_Pedido, p.FechaPedido, c.NombreCliente, s.NombreSucursal
    FROM C09Pedidos p
    INNER JOIN C09Clientes c ON p.ID_Cliente = c.ID_Cliente
    INNER JOIN C09Sucursales s ON p.ID_Sucursal = s.ID_Sucursal
    WHERE YEAR(p.FechaPedido) = @Anio
);
GO
```

```
SELECT * FROM fn_PedidosPorAnio(2024);
```

REPASANDO FUNCIONES



```
-- 5 Función con valor de tabla multisentencia (mTVF)
-- Objetivo: devolver los clientes junto con el promedio de sus pedidos y una categoría calculada.
CREATE FUNCTION fn_ClientesPromedio()
RETURNS @Tabla TABLE (
    ID_Cliente INT,
    NombreCliente VARCHAR(100),
    Promedio DECIMAL(10,2),
    Categoria VARCHAR(20)
)
AS
BEGIN
    INSERT INTO @Tabla
    SELECT c.ID_Cliente, c.NombreCliente,
        ISNULL(AVG(dp.Cantidad * dp.PrecioUnitario), 0) AS Promedio,
        CASE
            WHEN ISNULL(AVG(dp.Cantidad * dp.PrecioUnitario), 0) > 800 THEN 'Alta'
            WHEN ISNULL(AVG(dp.Cantidad * dp.PrecioUnitario), 0) BETWEEN 400 AND 800 THEN 'Media'
            ELSE 'Baja'
        END AS Categoria
    FROM C09Clientes c
    LEFT JOIN C09Pedidos p ON c.ID_Cliente = p.ID_Cliente
    LEFT JOIN DetalleC09Pedidos dp ON p.ID_Pedido = dp.ID_Pedido
    GROUP BY c.ID_Cliente, c.NombreCliente;
    RETURN;
END;

SELECT * FROM fn_ClientesPromedio();
```

CTE – Common Table Expression

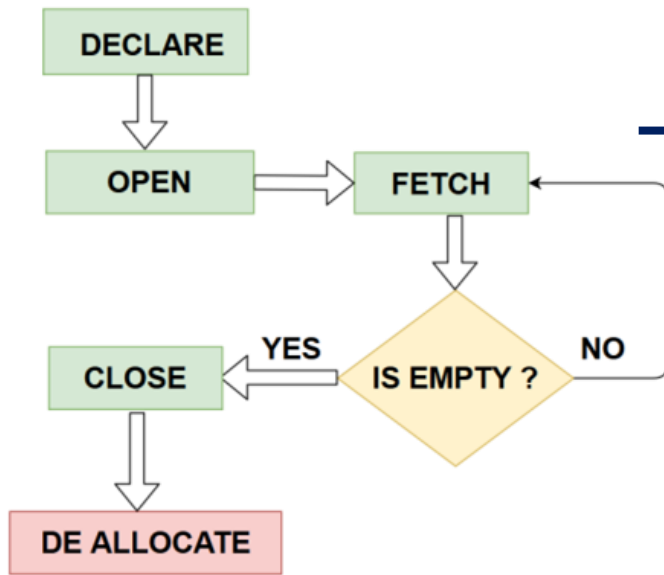
```
CREATE FUNCTION fn_Top5Clientes(@Año INT)
RETURNS @Resultado TABLE (ID_Cliente INT, Nombre NVARCHAR(100), Total MONEY)
AS
BEGIN
    -- Primer paso: calcular totales por cliente usando un CTE
    WITH VentasPorCliente AS (
        SELECT
            v.ID_Cliente, c.Nombre, SUM(v.Monto) AS Total
        FROM Ventas v
        INNER JOIN Clientes c ON v.ID_Cliente = c.ID_Cliente
        WHERE YEAR(v.Fecha) = @Año
        GROUP BY v.ID_Cliente, c.Nombre
    )
    -- Segundo paso: filtrar y devolver los 5 mejores
    INSERT INTO @Resultado
    SELECT TOP 5 ID_Cliente, Nombre, Total
    FROM VentasPorCliente
    WHERE Total > 0
    ORDER BY Total DESC;
    RETURN;
END;
```



Un CTE (Common Table Expression) es una consulta temporal nombrada que existe solo durante la ejecución de la sentencia siguiente.

CURSORES

Un cursor en SQL Server permite recorrer los resultados de una consulta fila por fila. Esto es útil cuando necesitas realizar operaciones detalladas sobre cada fila de un conjunto de resultados.



1. **Declarar el cursor:** Especificas la consulta que devolverá las filas a procesar.
2. **Abrir el cursor:** El cursor se abre para poder empezar a procesar las filas.
3. **Recuperar filas (FETCH):** Se recuperan las filas del cursor una a una.
4. **Procesar las filas:** Mientras el cursor tiene filas por procesar, puedes realizar las operaciones necesarias.
5. **Cerrar el cursor:** Una vez procesadas todas las filas, cierras el cursor.
6. **Liberar el cursor:** Finalmente, liberas los recursos del

```
-- Declaracion
DECLARE identificador CURSOR [ opciones ]
    FOR sentencia_select;

-- Abrir el cursor
OPEN identificador;

-- Variables que almacenarán los valores fila
FETCH [ opciones ] FROM identificador
    [ INTO lista_de_variables ];

-- Procesar las filas
WHILE @@FETCH_STATUS = 0
BEGIN
    -- Operaciones con las variables
    -- Leer fila siguiente
    FETCH NEXT FROM identificador
        [ INTO lista_de_variables ];
END;

-- Cerrar y liberar cursor
CLOSE identificador;
DEALLOCATE identificador;
```

-- recorrer una tabla de productos / reducir stock > 100

```
DECLARE @ID_Producto INT;  
DECLARE @CantidadStock INT;
```

-- Declarar el cursor para seleccionar productos con stock mayor a 100

```
DECLARE productos_cursor CURSOR FOR  
    SELECT ID_Producto, CantidadStock  
    FROM Producción.C11_Productos  
    WHERE CantidadStock > 100;
```

-- Abrir el cursor

```
OPEN productos_cursor;
```

-- Obtener el primer registro del cursor

```
FETCH NEXT FROM productos_cursor INTO @ID_Producto, @CantidadStock;
```

-- Procesar cada fila mientras haya filas disponibles

```
WHILE @@FETCH_STATUS = 0  
BEGIN
```

-- Reducir el stock en un 10% para productos con mucho stock

```
UPDATE Producción.C11_Productos  
SET CantidadStock = CantidadStock - (@CantidadStock * 0.1)  
WHERE ID_Producto = @ID_Producto;
```

-- Obtener el siguiente registro

```
FETCH NEXT FROM productos_cursor INTO @ID_Producto, @CantidadStock;
```

```
END;
```

-- Cerrar y liberar el cursor

```
CLOSE productos_cursor;  
DEALLOCATE productos_cursor;
```

DECLARACION

OPEN

FETCH
LEER 1° REGISTRO

Reduce Stock

FETCH
LEER SIGUIENTE
REGISTRO

@@FETCH_STATU
S
= 0

CLOSE

DEALLOCATE

S

N

CURSORES

Opciones principales de los cursores

1. Tipo de movimiento (Movimiento de filas):

***FORWARD_ONLY** (predeterminado)*

SCROLL

2. Tipo de acceso a datos (Modo de acceso a los datos):

***STATIC** – copia estática datos*

***DYNAMIC** (predeterminado) – refleja cambios en datos en tiempo real (ABM)*

***KEYSET** – refleja solo modificación de datos*

3. Concurrencia de datos (Permisos de modificación):

***READ_ONLY** (predeterminado)*

***SCROLL_LOCKS** – Bloquea filas para modificar*

***OPTIMISTIC** – No bloquea, si hay cambios da Error en tiempo de ejecución*

4. Combinación especial:

FAST_FORWARD

-- Declaracion

```
DECLARE identificador CURSOR [ opciones ]  
FOR sentencia_select;
```

-- Refleja los cambios en tiempo real y permite que las filas se actualicen.

```
DECLARE productos_cursor CURSOR DYNAMIC SCROLL_LOCKS FOR  
SELECT ID_Producto, CantidadStock FROM Producción.C11_Productos;
```

Movimiento	Tipo de datos	Concurrencia	Descripción
FORWARD_ONLY	STATIC	READ_ONLY	Movimiento hacia adelante, copia estática, sin modificación
SCROLL	DYNAMIC	SCROLL_LOCKS	Movimiento en cualquier dirección, cambios en tiempo real, con bloqueo
FAST_FORWARD	(implícito)	(implícito)	Movimiento hacia adelante, optimizado, sin modificación
FORWARD_ONLY	KEYSET	OPTIMISTIC	Movimiento hacia adelante, refleja cambios en filas seleccionadas, sin bloqueo

CURSORES

Opciones cláusula **FETCH**:

- **NEXT**: (*predeterminada*) Recupera la siguiente fila.
- **PRIOR**: Recupera la fila anterior. Requiere **SCROLL**.
- **FIRST**: Recupera la primera fila. Requiere **SCROLL**.
- **LAST**: Recupera la última fila. Requiere **SCROLL**.
- **ABSOLUTE n**: Recupera la fila número **n**.
Positivo, cuenta desde la primera fila;
Negativo, cuenta desde la última fila.
- **RELATIVE n**: Recupera la fila que está **n** posiciones desde la fila actual.
Positivo, cuenta hacia adelante;
Negativo, cuenta hacia atrás.

-- *Cláusula FETCH*

```
FETCH [ NEXT | PRIOR | FIRST | LAST |  
        ABSOLUTE n | RELATIVE n ]  
FROM nombre_cursor  
INTO variable1, variable2, ...;
```

CONTROL DE EJECUCION

Valores posibles de @@FETCH_STATUS:

- 0**: **FETCH** exitoso, fila recuperada.
- 1**: **FETCH** falló, no quedan más filas por recuperar.
- 2**: **FETCH** falló, fila ya no es válida (alterada).

*La cláusula **FETCH** en SQL Server se utiliza para recuperar filas de un cursor que ha sido previamente abierto. El cursor almacena un conjunto de resultados, y mediante el uso de **FETCH** puedes recuperar las filas de ese conjunto, una a una o en bloques, para procesarlas.*

TRANSACCIONES

Una transacción en SQL Server es un conjunto de operaciones SQL que se ejecutan como una unidad atómica, lo que significa que todas las operaciones deben completarse correctamente, o ninguna de ellas se aplicará. Esto asegura la consistencia de los datos.





TRANSACCIONES

Una transacción en SQL Server es un conjunto de operaciones SQL que se ejecutan como una unidad atómica, lo que significa que todas las operaciones deben completarse correctamente, o ninguna de ellas se aplicará. Esto asegura la consistencia de los datos.

- 1. BEGIN TRANSACTION:** Marca el inicio de una transacción.
- 2. COMMIT:** Confirma la transacción, haciendo que los cambios sean permanentes en la base de datos.
- 3. ROLLBACK:** Revierte (deshace) la transacción, deshaciendo todos los cambios realizados desde el inicio de la transacción.

*Requiere manejo de errores con
TRY-CATCH*

XACT_STATE():

1 – Transacción activa
-1 – Estado inválido
0 – No hay transacciones

```
BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE Producción.C11_Productos
    SET CantidadStock = CantidadStock - 50
    WHERE ID_Producto = 1;
    INSERT INTO Ventas.C11_RegistroVentas
    (CantidadVendida, FechaVenta)
    VALUES (1, 50, GETDATE());
    -- Si todo salió bien, confirma transacción
    COMMIT;
    PRINT 'Transacción completada con éxito.';
END TRY
BEGIN CATCH
    -- Si ocurre un error, revertir los cambios
    ROLLBACK;
    -- Mostrar mensaje de error
    PRINT 'Error en la transacción.';
    PRINT ERROR_MESSAGE();
END CATCH;
```



TRANSACCIONES

Tipo	Opción	Descripción / Uso recomendado
Punto intermedio	SAVE TRAN Punto1	Crea un punto de control dentro de la transacción. Permite rollback parcial.
Reversión parcial	ROLLBACK TRAN Punto1	Revierte hasta el punto guardado. Mantiene lo anterior.
Abortar autom	SET XACT_ABORT ON	Si ocurre un error, SQL Server hace ROLLBACK automático.
Aislamiento	SET TRANSACTION ISOLATION LEVEL ...	Define cómo se controlan las lecturas concurrentes para tratar el aislamiento
Chequeo de estado	@@TRANCOUNT	Devuelve la cantidad de transacciones activas (útil para detectar anidamiento).

```
SET XACT_ABORT ON; -- Opcional: fuerza rollback automático ante error
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED; -- Nivel por defecto
```

```
BEGIN TRY
    BEGIN TRANSACTION NombreTransaccion -- Opcional: se puede nombrar
```

```
-- Sentencias SQL aquí
INSERT INTO ...
UPDATE ...
DELETE ...
```

```
    COMMIT TRANSACTION NombreTransaccion
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION NombreTransaccion
    PRINT 'Error: ' + ERROR_MESSAGE()
END CATCH
```



TRANSACCIONES

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

BEGIN TRAN

UPDATE Inventario SET Stock = Stock - 10 WHERE ID_Producto = 100;

INSERT INTO Ventas VALUES (100, 10, GETDATE());

COMMIT TRAN

Nivel	Qué bloquea	Qué permite	Comando
READ UNCOMMITTED	Nada	Lee datos no confirmados	SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
READ COMMITTED	Lecturas sucias	Lecturas no repetibles	SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
REPEATABLE READ	Lecturas sucias y no repetibles	Phantom reads	SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SERIALIZABLE	Todos los anteriores	Ninguno	SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SNAPSHOT	Utiliza versión de fila	Requiere habilitar snapshot isolation en la BD	SET TRANSACTION ISOLATION LEVEL SNAPSHOT;



TRANSACCIONES

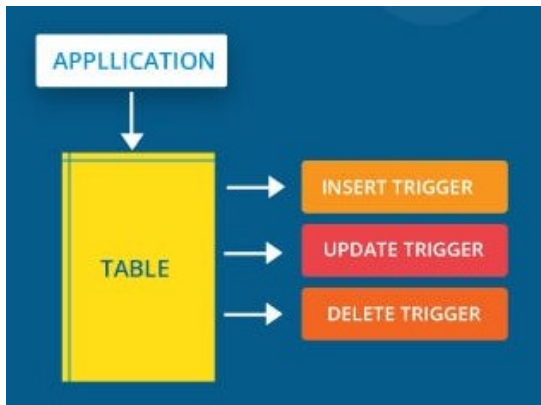
Tipos de control de transacciones en SQL Server: Se puede ejecutar transacciones de tres formas diferentes:

- **Autocommit (automáticas)**
- **Implícitas**
- **Explícitas**

```
-- Cada sentencia es una transacción completa
INSERT INTO Productos VALUES (1, 'Mouse', 2000); --
COMMIT automático
UPDATE Productos SET Precio = 2500 WHERE ID = 1; --
COMMIT automático

-- Transacción Implícita
SET IMPLICIT_TRANSACTIONS ON;
INSERT INTO Productos VALUES (2, 'Teclado', 3000); --
Inicia transacción automáticamente
-- Permanece abierta hasta COMMIT o ROLLBACK
COMMIT TRAN; -- Cierre manual

-- Transacción Explícita
BEGIN TRAN
    INSERT INTO Productos VALUES (3, 'Monitor', 7000);
    UPDATE Stock SET Cantidad = Cantidad - 1 WHERE
ID_Producto = 3;
COMMIT TRAN
```



TRIGGERS

Un trigger es un objeto especial en SQL Server que se ejecuta automáticamente en respuesta a un evento específico en una tabla o vista. Los triggers se utilizan para automatizar tareas o aplicar reglas de negocio cuando ocurren eventos como INSERT, UPDATE o DELETE.

Tipos de triggers en SQL Server:

1. **AFTER (después del evento):** El trigger se ejecuta **después** de que ocurra la acción de **INSERT, UPDATE, o DELETE**.
2. **INSTEAD OF (en lugar del evento):** El trigger se ejecuta **en lugar de** la acción que lo invoca.

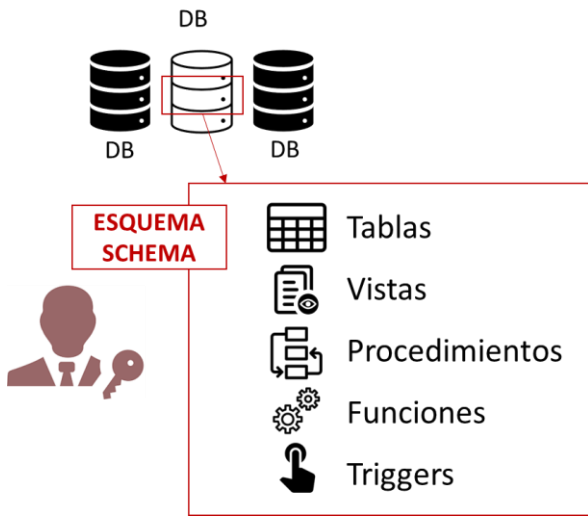


INSERTED

DELETED

```
CREATE TRIGGER trg_AfterInsert_Ventas
ON Ventas.C11_RegistroVentas
AFTER INSERT
AS
BEGIN
    -- Insertar en la tabla de auditoría un
    -- registro con los datos insertados
    INSERT INTO Auditoria.AuditoriaVentas
    (ID_Producto, CantidadVendida, FechaVenta)
    SELECT ID_Producto, CantidadVendida,
    FechaVenta
    FROM INSERTED;

    PRINT 'Registro insertado en la tabla de
    auditoría.';
END;
```



ESQUEMAS

Un esquema en SQL Server es una forma de organizar objetos de base de datos (como tablas, vistas, procedimientos almacenados, funciones, etc.) en contenedores lógicos. Esquemas permiten agrupar objetos relacionados para facilitar su administración, control de permisos y mantenimiento.

Beneficios de usar esquemas:

1. **Organización:** ordenamiento de los objetos dentro de una base de datos.
2. **Seguridad:** Facilitan la asignación de permisos.
3. **Flexibilidad:** Usuarios pueden utilizar sus propios objetos sin interferir sobre otros objetos de la base de datos.
4. **Despliegue controlado:** Mover objetos entre esquemas sin renombrarlos.

```
CREATE SCHEMA nombre_esquema;
```

```
CREATE TABLE nombre_esquema.nombre_tabla (  
    columna1 INT,  
    columna2 VARCHAR(100)  
);
```

```
GRANT SELECT ON schema::nombre_esquema TO usuario;
```



SINONIMOS

Un sinónimo en SQL Server es un alias que creas para referirte a otro objeto de la base de datos, como una tabla, vista, procedimiento almacenado, función o incluso un objeto ubicado en otra base de datos o servidor. El sinónimo proporciona una abstracción sobre el nombre y la ubicación del objeto, lo que facilita el acceso y mejora la flexibilidad.

Beneficios de usar esquemas:

1. **Simplificación:** nombres cortos y fáciles de recordar.
2. **Independencia de ubicación:** no hace falta cambiar las consultas que hacen referencia al objeto (solo actualizar el sinónimo).
3. **Abstracción:** cambiar estructura de la base de datos sin afectar a las aplicaciones que la consumen.

```
CREATE SYNONYM nombre_sinonimo  
FOR esquema.objeto;
```

```
CREATE SYNONYM ProductosAlias  
FOR Producción.C11_Productos;
```

```
SELECT * FROM ProductosAlias;
```

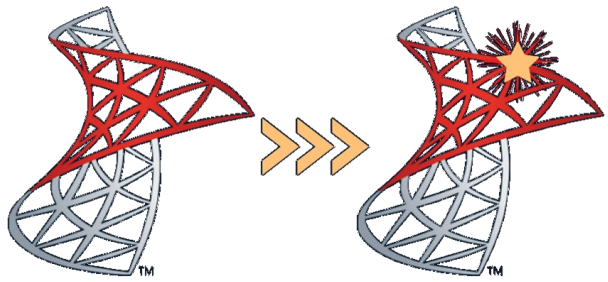


Tablas:

Locales (en la misma base de datos).
En otra base de datos.
En otro servidor.



Vistas: Locales o en bases de datos remotas.
Procedimientos almacenados.
Funciones definidas por el usuario.



SQL Dinámico

El SQL dinámico es una técnica en SQL Server que permite generar y ejecutar consultas SQL en tiempo de ejecución. A diferencia de las consultas SQL estáticas, que están completamente definidas antes de su ejecución, el SQL dinámico construye consultas sobre la marcha, basándose en variables o parámetros.

Pasos:

- 1. Construcción:** Durante la ejecución de una operación, transacción o programas, se construye una cadena de texto que representa la consulta que se quiere ejecutar. Incluye partes variables o parámetros que toman valores específicos.
- 2. Ejecución:** SQL Server proporciona la instrucción EXEC o la función **sp_executesql** para ejecutar la cadena generada dinámicamente.

```
--Consulta con SQL dinámico:
DECLARE @columna NVARCHAR(50) = 'NombreProducto';
DECLARE @valor NVARCHAR(100) = 'ProductoA';

-- Construcción dinámica de la consulta
DECLARE @sql NVARCHAR(MAX);
SET @sql = 'SELECT * FROM Producción.C11_Productos
          WHERE ' + @columna + ' = @valor';

-- Ejecutar la consulta dinámica
EXEC sp_executesql @sql, N'@valor NVARCHAR(100)', @valor;
```

```
DECLARE @sql NVARCHAR(MAX);
SET @sql = 'SELECT * FROM Producción.C11_Productos WHERE ' +
          @columna + ' = ''' + @valor + '''';

-- Ejecutar la consulta dinámica usando EXEC
EXEC(@sql);
```

SQL - *Structured Query Language*



Bibliografía:

Introducción a los Sistemas de Bases de Datos - C. J. Date – 7° Edición

- **CAPÍTULO 8: Integridad**
- **CAPÍTULO 13: Modelado semántico**
- **CAPÍTULO 20: Bases de datos distribuidas**

Sistemas de Bases de Datos – Conceptos Fundamentales - Elmasri / Navathe – 5° Edición.

- **Capítulo 9: Introducción a las técnicas de programación SQL.**
- **Capítulo 20: Modelos de datos mejorados para aplicaciones avanzadas**

Microsoft Docs – SQL Server: SQL Dynamic

Próximo encuentro



Clase 11: Normalización y Modelo ER

Dependencias funcionales: concepto y usos.

Identificación – aplicación en el diseño de BD

Ejercitación práctica..